Al-Razi University

USB SECURITY MONITOR

Course: Internet of Things (IoT)

Semester: Level 4 - Semester 1 (2026)

Submission Date: 10 October 2026

Team Members:

- آية عبدالملك علي المسوري - 30
- أنوار بشير محمد الجشاعة - 26
- كوثر توفيق يحيى الدغار - 118
- اسماء محمد محسن الانسي - 16

Instructor:
Eng. Majd Al-Suraihi

2. Abstract

Uncontrolled USB flash-drive use can introduce untrusted storage devices into shared computers. This project implements USB Security Monitor, a low-cost demonstration system that identifies USB mass-storage devices, compares their Windows Plug and Play instance identifiers against a local trusted-device list, and provides immediate visual and on-screen alerts. The prototype combines a Windows 10 laptop, a PowerShell Windows Forms interface with a background C# / WMI scanning component, an ESP32 development board, and green/red LEDs connected through current-limiting resistors on a breadboard. The Windows application polls connected USB storage devices, records newly observed device events with timestamps, and sends serial commands to the ESP32 at 115200 baud. A registered device produces a green indicator; an unregistered device produces a red indicator and an audible alert from the computer. Removing the last USB drive returns the indicators to the idle state. The implementation was tested with a registered flash drive and a different unregistered flash drive; the graphical interface, serial indication and event logging worked in the observed demonstrations. The project provides identification and alerting only: it does not block USB access or detect malware. Future extensions may include integrated device registration, antivirus scanning and policy-based storage control.

3. Introduction

3.1 Background and Motivation

Portable USB storage remains convenient for moving files between systems, yet users may insert devices of unknown ownership without knowing whether those devices are trusted. This project introduces an observable USB-checking workflow appropriate for laboratory demonstration and awareness training. Instead of implementing a kernel driver, the team integrates Windows device information with an inexpensive ESP32 status indicator and a local event log.

3.2 Problem Statement

A standard Windows desktop does not provide the project-specific allowlist indication required for a classroom demonstration: the user may not know if a newly inserted flash drive belongs to a predefined approved set. The design challenge is to detect storage-device attachment, classify it against a configurable local list, notify the user clearly, and record the event without interrupting normal workstation use.

3.3 Project Objectives

- Identify newly connected USB mass-storage devices on a Windows computer using Plug and Play instance identifiers.
- Compare each detected identifier to a local trusted list stored in TrustedUSB.txt.
- Provide green/red LED indications through an ESP32 and an alert in a desktop GUI.
- Record connection time, classification, device type, and instance identifier in a local event log.
- Allow a new test flash drive to become trusted by adding its confirmed identifier to the allowlist.
- Maintain usable GUI controls while device detection runs in a separate background thread.

3.4 Scope and Limitations

The tested build operates on a Windows laptop and focuses on the device name "USB Mass Storage Device". The GUI recognizes identifiers with a USB\VID_ prefix, and the allowlist is compared against the full instance identifier. The project supports local monitoring, classification, logging, serial LED status and a computer-generated beep. It does not block access to unregistered drives, disinfect files, inspect USB traffic, or guarantee that an identifier cannot be spoofed. Devices already attached when monitoring

starts may require a safe disconnect/reconnect to appear as new events. No numerical latency, current-consumption or attack-resistance benchmark was collected.

4. System Overview

4.1 High-Level Description

A PowerShell desktop application queries Windows for connected USB storage devices. It compares the detected Plug and Play instance identifiers with entries in TrustedUSB.txt and evaluates the system state. A serial command is sent through the USB cable to the ESP32, which controls the green and red LEDs. The interface presents alert status, counters and a connection history. The laptop remains the decision-making host; the ESP32 functions as an output controller.

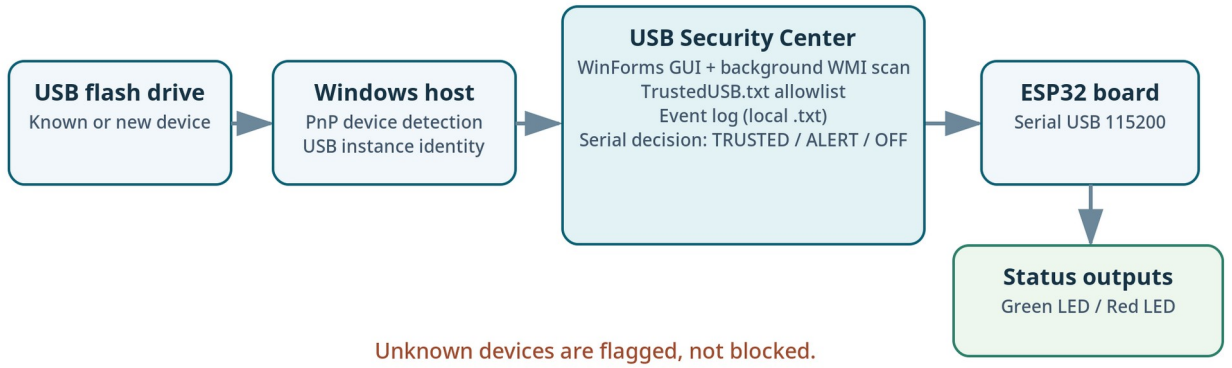


Figure 1. Implemented high-level system block diagram. The system alerts but does not block access.

4.2 Functional Description

- Detection: Windows Management Instrumentation (WMI) reports connected PnP devices; the application selects working USB mass-storage devices.
- Identification: each new PNPDeviceID is compared with the case-insensitive set of full IDs in TrustedUSB.txt.
- Decision: if any current device is unknown, overall output is ALERT; otherwise TRUSTED when known drives exist, or OFF when none are present.
- Actuation: serial messages TRUSTED, ALERT or OFF drive the external LED indicators.
- Notification and logging: a Windows Forms GUI displays status and cumulative attachment counters; a text file stores timestamped device events.

4.3 Use-Case Scenario

A student starts the GUI, confirms the ESP32 serial port (COM10 during the reported test) and clicks Start Monitoring. After reconnecting a registered flash drive, the program displays a registered event and illuminates the green LED. Inserting a different drive whose ID is not in the trusted list changes the status to ALERT, illuminates the red LED and produces a PC beep. When the unregistered drive is safely removed, the status returns to the remaining connected device or OFF if no monitored storage remains. The history retains recorded events.

5. Hardware Design

5.1 Components List

COMPONENT	SPECIFICATION / ROLE	QTY
ESP32 development board	ESP32 Dev Module, USB programmable, 3.3 V GPIO	1
Breadboard	Solderless prototyping board	1
Green LED	Status: registered USB device	1
Red LED	Status: unregistered USB device	1
LED resistors	Series current-limiting resistors (verify actual values)	2
Jumper wires	Board-to-breadboard connections	Several
USB data/power cable	Laptop to ESP32 communication and power	1
Windows laptop	PnP detection, application, log and serial host	1
Piezo buzzer	Present in prototype; ESP32-driven sound not validated	1

5.2 Component Justification

The ESP32 is inexpensive, widely supported by Arduino IDE, and exposes GPIO pins for direct visual feedback. A solderless breadboard supports rapid changes while testing. The two LEDs communicate device classification at a glance without requiring continuous focus on the computer screen. Using the existing Windows laptop avoids a dedicated USB-host module: identification happens on the host that already enumerates the inserted flash drive.

5.3 Block Diagram

See Figure 1 in Section 4.1. The input is the Windows USB-device event, processing is local allowlist classification, and outputs are GUI status, an event log, an optional PC beep, and serial-controlled LEDs.

5.4 Circuit Diagram

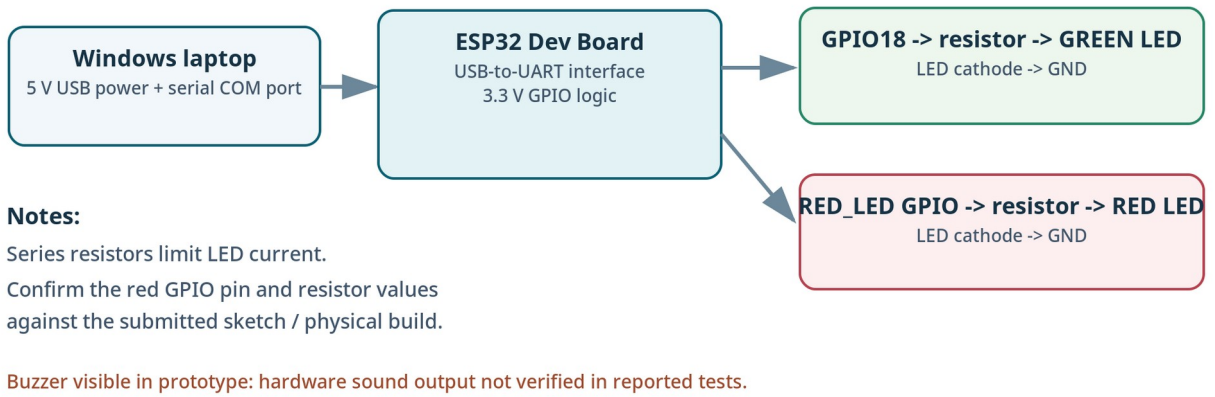


Figure 2. Functional wiring schematic. Red-LED GPIO number and measured resistor values require final checking against the physical sketch before declaring an exact pinout.

Wiring record: Green LED is defined as GPIO18 in the demonstrated firmware. The exact RED_LED pin number is not legible in the supplied record; avoid entering an unverified number in the final schematic.

5.5 Power Considerations

The ESP32 board is powered from the laptop USB port (nominal USB 5 V input), while the chip GPIO logic operates at 3.3 V. External LEDs should be powered only through correctly selected current-limiting resistors; the ESP32 GPIO should never be driven with 5 V logic. The prototype uses no external mains connection or battery. Total board current was not measured; therefore a numerical current estimate is not reported as a test result. The USB cable provides a shared host connection for power and serial data.

6. Software Design

6.1 Development Environment

LAYER	TOOLS / LANGUAGES
ESP32 firmware	Arduino IDE 2.3.10; Arduino-style C/C++; digital GPIO control.
Desktop controller	Windows PowerShell 5.1; Windows Forms GUI; embedded C# background scanner.
Windows device information	WMI System.Management; Win32_PnPEntity records; PNPDeviceID.
Serial link	System.IO.Ports.SerialPort, 115200 baud, newline-delimited text.
Configuration and audit	TrustedUSB.txt; USB_Security_Log.txt; local desktop files.

6.2 Software Architecture

The implemented V2 desktop GUI starts a dedicated background scanning thread so that slow Windows device queries do not block the interface. The worker loads trusted identifiers, opens the selected COM port, waits briefly for ESP32 reset after serial connection, and then polls connected USB storage at approximately 1.4-second intervals. New identifiers are classified and queued to the GUI, which updates labels, counters and the event table. The worker issues serial commands only as connection state changes. On stopping, it attempts to send OFF and closes the port. Arduino firmware receives newline-terminated commands and uses digitalWrite() to set LED outputs.

6.3 Flowchart / State Diagram

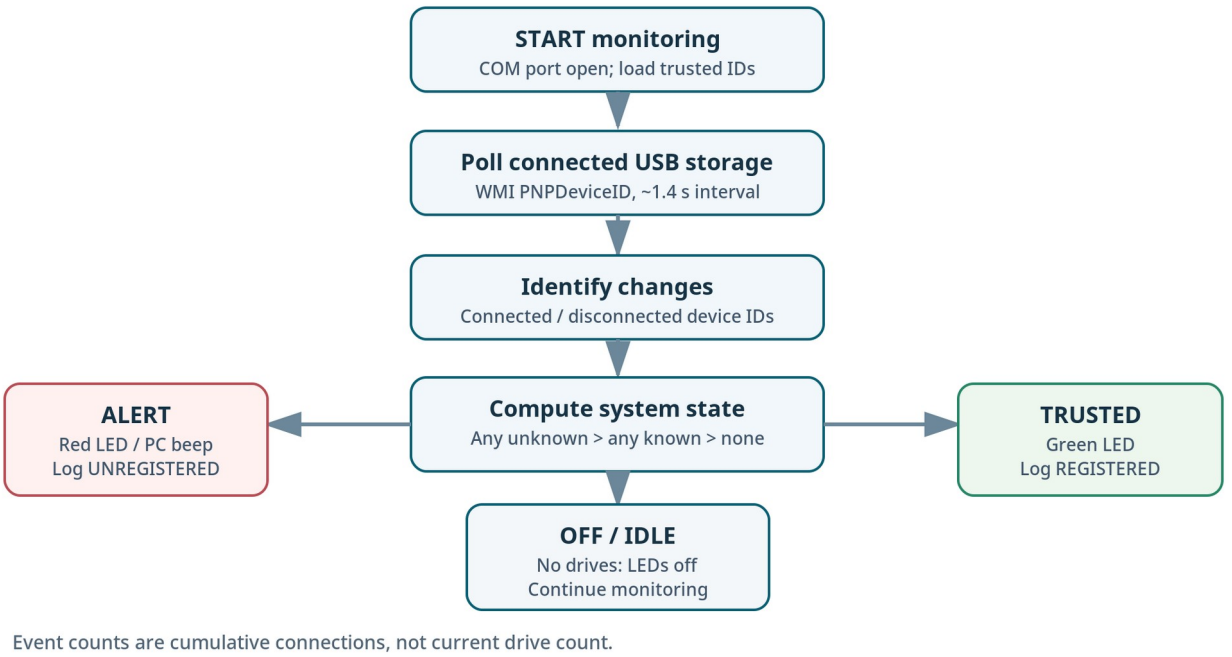


Figure 3. Monitoring state logic (unregistered drives take priority if multiple drives are present).

6.4 Algorithms

Allowlist matching. Load one complete USB instance ID per line from TrustedUSB.txt, and store IDs in a case-insensitive set. After each scan, calculate newIds = currentIds - previousIds and removedIds = previousIds - currentIds. For each new ID, classify it as REGISTERED if it is in the trusted set; otherwise UNREGISTERED. Log the timestamp and identifier, and increment the appropriate event counter. Then derive the overall output state from all currently present detected storage devices, with the priority ALERT > TRUSTED > OFF.

This is identity-based matching, not cryptographic authentication. Windows-supplied identifiers can vary across devices and can potentially be imitated. No virus analysis is performed by this algorithm.

7. System Integration

The PC connects to the ESP32 through the board USB connector. Windows enumerates a serial COM port; COM10 was used in the demonstration, though this may change on another machine. The host sends simple ASCII text messages (TRUSTED, ALERT, OFF) terminated by a newline over a 115200-baud serial connection. The ESP32 changes GPIO output levels accordingly. Serial Monitor and the desktop application must not hold the same COM port simultaneously. A delay after opening the port accommodates possible ESP32 reset. The background polling strategy provides human-visible near-real-time alerts rather than hard real-time guarantees.

The tested application expects its trusted-list and log files at the desktop (or detects the trusted file next to the script in a supported location). If a new USB device must be approved, the operator verifies its exact identifier and adds a new line to TrustedUSB.txt before restarting monitoring. No Windows device restrictions are enabled.

8. Testing and Validation

8.1 Testing Strategy

Validation followed a progression from individual LED upload tests to firmware serial-command tests and then full system tests with known and unknown flash drives. The team checked output changes physically (green/red LEDs), on the GUI (classification message and event counters), and in the text event log. Negative testing included observing COM-port contention during firmware upload and confirming that stopping the serial-owning application frees the port.

8.2 Test Cases

ID	INPUT / CONDITION	EXPECTED OUTPUT	OBSERVED OUTPUT	RESULT
TC-01	Upload a basic GPIO18 blink sketch	Green LED blinks	LED observed turning on and off	PASS
TC-02	Connect an allowlisted USB drive	Green LED; REGISTERED entry	Green LED and registered GUI/log entry observed	PASS
TC-03	Connect a different, unlisted USB drive	Red LED; alert and log entry	Red LED and UNREGISTERED entry observed	PASS
TC-04	Safely remove the last monitored drive	Status OFF; LED goes out	Indicator turned off after removal	PASS
TC-05	Open event-history view in revised GUI	History is displayed without freezing	History window displayed; responsive in reported test	PASS
TC-06	Add a verified drive ID to trusted list, retest	New drive subsequently classifies as trusted	User confirmed new registration and green indicator	PASS
TC-07	Upload Arduino sketch while GUI owns COM10	Upload fails (port busy)	Access denied / port busy observed	EXPECTED CONFLICT
TC-08	Scan flash-drive files for malware	Detect and report malware	Not implemented or tested	NOT TESTED

8.3 Hardware and Software Testing

Early uploads failed because the ESP32 board package was not installed in Arduino IDE and later because Windows COM10 was already open in another program. Installing the Espressif board support package resolved compilation/toolchain availability. After closing the monitor application / Serial Monitor, upload succeeded and the ESP32 firmware remained stored in flash memory. The first desktop GUI version had slow control response during device polling; the revised version moved Windows device queries to a background worker and the in-application event history was verified to open responsively.

Evidence boundary: Pass labels describe the observed classroom demonstration, not a formal performance benchmark. Physical buzzer operation, malware identification and USB blocking were not demonstrated as completed features.

9. Results and Discussion

The prototype successfully demonstrated distinction between registered and unregistered USB mass-storage devices and linked the classification to physical LED indicators and a readable GUI history. The supplied images and on-device trial showed the green state for an approved drive, the red state for an unknown drive, and an idle/off state after removal. The application also retained chronological records containing a timestamp, state, device name and full Windows instance identifier. Repeated insertions

appear as separate logged events, so the counters show cumulative events rather than the number of USB drives currently present.

The solution is attractive as a learning prototype because it combines host-side enumeration and embedded output control with minimal hardware. The architecture deliberately favors a transparent alert mechanism over invasive operating-system policy changes. Consequently, the security effect is limited to user awareness and operator response. An unregistered drive may remain accessible to Windows, and its presence is not proof of malware.

10. Challenges and Solutions

CHALLENGE	SOLUTION / LESSON
ESP32 target initially unavailable	Installed the esp32 by Espressif Systems board package; selected ESP32 Dev Module.
Firmware upload failed with COM10 access denied	Stopped the PowerShell GUI / closed Serial Monitor before uploading; one serial owner at a time.
PowerShell script execution restricted	Used a temporary process-only execution policy for the known local script, rather than changing the system-wide policy.
GUI controls delayed by Windows device queries	Used a background device-scanning worker and UI event queue in the revised interface.
Need to add an approved flash drive	Recorded its exact PnP instance identifier in TrustedUSB.txt and restarted monitoring.
LED-only indication insufficient to interpret the event	Added on-screen status, event history, timestamps and cumulative counters.

11. Conclusion

The team developed and demonstrated a working USB identification and alert prototype using Windows, a PowerShell GUI and an ESP32 development board. The primary objectives of classifying connected USB mass-storage devices, recording attach events, and showing green/red status were achieved in the reported hands-on tests. Reliability outside the tested Windows computer, a larger collection of drives and hostile USB hardware remains unverified. The system should be described as a monitoring and awareness tool - not a USB access-control product or antivirus engine.

12. Future Work

- Integrate a safe "Register selected USB" workflow with user confirmation, duplicate prevention and an audit entry.
- Provide a dedicated device-status panel showing currently connected drives separately from cumulative events.
- Use an installed antivirus engine to perform an optional on-demand scan; display its real scan result rather than a simulated status.
- Add optional policy-based blocking after careful testing, administrator approval and a documented rollback procedure.
- Verify the buzzer wiring, electrical drive limits and firmware behavior; then add an optional physical alert.
- Improve robustness with serial-port auto-discovery, device-identity validation, packaging, and longer repeated tests.

13. References

[1] Espressif Systems, "ESP32-WROOM-32 Datasheet," official technical documentation. https://documentation.espressif.com/esp32-wroom-32_datasheet_en.html

[2] Microsoft, "Win32_PnPEntity class," Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/cimwin32prov/win32-pnpentity

[3] Microsoft, "SerialPort Class (System.IO.Ports)," Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/system.io.ports.serialport

[4] Arduino, "digitalWrite()," Arduino Language Reference. https://docs.arduino.cc/language-reference/en/functions/digital-io/digitalwrite/

[5] Project development materials: USB_Security_GUI.ps1 (V2), Arduino sketch and experimental observations provided during development (October 2026).

14. Appendices

Appendix A: Source Code

Recommended upload to the course form: a ZIP containing the final ESP32 Arduino .ino sketch, the tested Windows desktop GUI source USB_Security_GUI.ps1, the starter START_USB_SECURITY.bat, and a README. Exclude personal logs, physical device identifiers and any confidential workstation data if the repository is public. The full Arduino firmware file is not included in the provided report materials; it must be added from the development laptop before submission.

Key serial interface expected by the tested GUI:

```
Baud rate: 115200
Commands sent: TRUSTED, ALERT, OFF (newline terminated)
TRUSTED -> green on / red off
ALERT -> red on / green off
OFF -> both LEDs off
```

Core host-side scan criteria (from the tested V2 application):

```
Win32_PnPEntity.Name == 'USB Mass Storage Device'
PNPDeviceID starts with USB\VID_
ConfigManagerErrorCode == 0
Allowlist: full PNPDeviceID exact match (case-insensitive)
Current state: unknown present -> ALERT; else known present -> TRUSTED; else OFF
```

GitHub Repository: https://github.com/AyaAlMaswari/USB-Security-Monitor

Appendix B: Additional Schematics/Figures

Figure 2 serves as the wiring-level schematic, while Figure 3 documents the software state transitions. Two prototype photographs are reproduced below from the project development record; they are demonstrations of hardware setup and the green indicator, not evidence of virus detection.

A. ESP32 prototyping



B. Registered drive / green LED

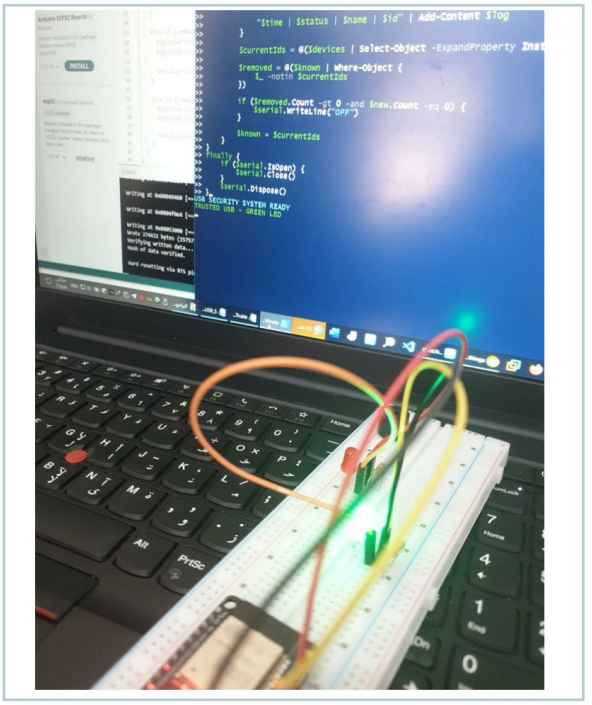


Figure B1. Prototyping hardware and the registered-device green indication (user-provided photographs).

Additional submission note: the Google Form requests the source-code archive and PDF project report as separate attachments. This document is the report; it is not a substitute for the source archive.